

Atelier Big Data

Apache Airflow pour l'orchestration des pipelines Big Data

Filière : II-BDDC

Module : Big Data

Outils : Docker, Docker Compose, Apache Airflow, PythonOperator

Objectifs de l'atelier

À la fin de cet atelier, vous devez être capables de :

- comprendre le rôle d'Apache Airflow dans les pipelines Big Data ;
- expliquer ce qu'est un DAG ;
- créer un pipeline simple avec plusieurs tâches ;
- utiliser `PythonOperator` pour exécuter des fonctions Python ;
- simuler les étapes d'un pipeline Big Data ;
- définir l'ordre d'exécution des tâches ;
- exécuter un DAG depuis l'interface Web ;
- consulter les logs d'une tâche ;
- comprendre la planification automatique ;
- comprendre la gestion des erreurs et des reprises ;
- créer des dépendances parallèles dans un DAG ;
- expliquer pourquoi Airflow est important dans une architecture Data Engineering.

1 Introduction

Dans les projets Big Data, les traitements de données sont rarement exécutés en une seule étape.

Un pipeline de données peut contenir plusieurs étapes :

1. récupérer les données depuis différentes sources ;
2. vérifier que les données sont disponibles ;
3. stocker les données brutes ;
4. nettoyer les données ;
5. transformer les données ;
6. calculer des indicateurs ;
7. stocker les résultats ;
8. générer un rapport ;
9. alimenter un tableau de bord.

Ces étapes doivent être exécutées dans un ordre précis. Si une étape échoue, il faut pouvoir identifier l'erreur, consulter les logs et relancer le traitement.

Apache Airflow permet d'automatiser et de surveiller ce type de pipeline.

Définition

Apache Airflow est une plateforme d'orchestration de workflows. Il permet de définir, planifier, exécuter et surveiller des pipelines sous forme de DAGs.

1.1 Idée principale

Apache Airflow ne remplace pas les outils de traitement comme Spark, Hadoop, Flink, Hive ou Kafka.

Son rôle principal est d'organiser les traitements.

Apache Airflow = chef d'orchestre des workflows

Par exemple, Airflow peut organiser un pipeline comme suit :

Ingestion → Stockage brut → Validation → Transformation → Chargement →
Rapport

2 Concepts essentiels

2.1 Utilisation d'Apache Airflow dans les pipelines Big Data

Dans une architecture Big Data, les données passent généralement par plusieurs étapes avant d'être utilisées pour l'analyse ou la prise de décision.

Un pipeline Big Data peut contenir les étapes suivantes :

1. ingestion des données depuis des fichiers, APIs, bases de données ou applications ;
2. stockage des données brutes dans un Data Lake ;
3. validation de la qualité des données ;
4. nettoyage et transformation des données ;
5. traitement analytique avec Python, Spark ou Flink ;
6. chargement des résultats dans un Data Warehouse ;
7. visualisation avec des outils de Business Intelligence ;
8. génération de rapports ou de tableaux de bord.

Dans ce type d'architecture, Apache Airflow joue le rôle d'orchestrateur.

Cela signifie qu'il ne réalise pas forcément lui-même le traitement lourd des données, mais il organise l'exécution des différentes étapes.

Sources → Data Lake → Traitement → Data Warehouse → Dashboard

Airflow permet de répondre aux questions suivantes :

- Quelle tâche doit être exécutée en premier ?
- Quelle tâche dépend d'une autre tâche ?
- Que faire si une tâche échoue ?
- Comment relancer automatiquement une tâche ?
- Comment consulter les logs ?
- Comment automatiser l'exécution quotidienne d'un pipeline ?

2.2 Importance d'Airflow dans un projet Big Data

Dans un projet Big Data réel, il peut y avoir plusieurs outils différents :

- un outil de stockage comme HDFS, MinIO ou Amazon S3 ;
- un moteur de traitement comme Spark ou Flink ;
- une base analytique comme PostgreSQL, Hive ou BigQuery ;
- un outil de visualisation comme Superset, Power BI ou Tableau.

Sans orchestrateur, il faut lancer manuellement chaque étape ou écrire des scripts difficiles à maintenir.

Avec Apache Airflow, le pipeline devient :

- **automatisé** : les tâches peuvent être exécutées selon une planification ;
- **supervisé** : l'état de chaque tâche est visible dans l'interface Web ;
- **traçable** : les logs et l'historique des exécutions sont conservés ;
- **fiable** : les erreurs sont détectées et certaines tâches peuvent être relancées ;
- **maintenable** : le pipeline est décrit sous forme de code Python ;
- **compréhensible** : la vue Graph permet de visualiser les dépendances.

Remarque

Le rôle d'Airflow n'est pas de remplacer les outils Big Data. Son rôle est de les coordonner dans un pipeline cohérent et automatisé.

2.3 Exemple simple d'utilisation

Dans un projet Big Data, Airflow peut orchestrer le scénario suivant :

Fichier CSV → Data Lake → Traitement Python/Spark → Base analytique →
Dashboard

Étape	Exemple de tâche orchestrée par Airflow
Ingestion	Récupérer ou générer un fichier de ventes.
Stockage	Copier ou simuler le stockage du fichier dans une zone brute.
Validation	Vérifier que les données sont présentes et cohérentes.
Transformation	Nettoyer les données et créer de nouvelles colonnes.
Traitement analytique	Calculer des indicateurs comme le chiffre d'affaires par ville.
Chargement	Stocker les résultats dans une base analytique ou un fichier résultat.
Rapport	Générer un résumé final du pipeline.

2.4 DAG

Définition

Un DAG, ou Directed Acyclic Graph, est un graphe orienté sans cycle. Dans Airflow, un DAG représente un pipeline composé de plusieurs tâches.

Un DAG définit :

- les tâches à exécuter ;
- l'ordre d'exécution ;
- la fréquence d'exécution ;
- le comportement en cas d'échec.

2.5 Task

Définition

Une tâche est une étape individuelle dans un pipeline Airflow.

Exemples de tâches :

- vérifier l'existence d'un fichier ;
- générer un fichier CSV ;
- valider un schéma de données ;
- nettoyer les données ;
- calculer des indicateurs ;
- générer un rapport ;
- lancer un traitement Spark.

2.6 Operator

Définition

Un Operator est un modèle de tâche. Il indique à Airflow comment exécuter une action.

Operator	Rôle
<code>PythonOperator</code>	Exécuter une fonction Python.
<code>BashOperator</code>	Exécuter une commande Bash.
<code>EmptyOperator</code>	Créer une tâche vide pour structurer un DAG.

Dans cet atelier, nous allons utiliser principalement `PythonOperator`, car il permet de simuler facilement les étapes d'un pipeline Big Data avec du code Python.

2.7 Scheduler

Le scheduler est le composant qui décide quand les DAGs et les tâches doivent être exécutés.

2.8 Worker

Le worker est le composant qui exécute réellement les tâches.

2.9 Web UI

L'interface Web d'Airflow permet de :

- voir la liste des DAGs ;
- activer ou désactiver un DAG ;

- déclencher un DAG manuellement ;
- visualiser le graphe des tâches ;
- consulter les logs ;
- voir les erreurs ;
- relancer une tâche échouée.

3 Installation d'Apache Airflow avec Docker

3.1 Prérequis

Avant de commencer, il faut avoir installé :

- Docker Desktop ;
- Docker Compose ;
- Visual Studio Code ou un autre éditeur ;
- un navigateur Web.

Remarque

Il est recommandé d'avoir au moins 4 Go de mémoire disponible pour lancer Airflow correctement avec Docker.

3.2 Créer le dossier de travail

Créer un dossier nommé `atelier-airflow`.

```
mkdir atelier-airflow
cd atelier-airflow
```

3.3 Créer les dossiers nécessaires

Créer les dossiers utilisés par Airflow :

```
mkdir dags
mkdir logs
mkdir plugins
mkdir config
mkdir data
```

3.4 Créer le fichier Docker Compose

Dans le dossier `atelier-airflow`, créer le fichier suivant :

`docker-compose.yaml`

Dossier	Rôle
dags	Contient les fichiers Python des DAGs.
logs	Contient les logs d'exécution.
plugins	Contient les extensions personnalisées.
config	Contient des fichiers de configuration.
data	Contient les fichiers de données manipulés par les DAGs.

Ajouter le contenu suivant :

```
services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow

  airflow-webserver:
    image: apache/airflow:2.8.1
    depends_on:
      - postgres
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:
        airflow@postgres/airflow
      AIRFLOW__CORE__LOAD_EXAMPLES: "false"
    volumes:
      - ./dags:/opt/airflow/dags
      - ./logs:/opt/airflow/logs
      - ./plugins:/opt/airflow/plugins
      - ./data:/opt/airflow/data
    ports:
      - "8080:8080"
    command: webserver

  airflow-scheduler:
    image: apache/airflow:2.8.1
    depends_on:
      - airflow-webserver
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:
        airflow@postgres/airflow
```

```
AIRFLOW__CORE__LOAD_EXAMPLES: "false"
volumes:
- ./dags:/opt/airflow/dags
- ./logs:/opt/airflow/logs
- ./plugins:/opt/airflow/plugins
- ./data:/opt/airflow/data
command: scheduler
```

3.5 Initialiser Airflow

Exécuter la commande suivante :

```
docker compose run airflow-webserver airflow db init
```

Cette commande initialise l'environnement Airflow.

3.6 Créer un utilisateur Airflow

Créer l'utilisateur administrateur pour accéder à l'interface Web :

```
docker compose run airflow-webserver airflow users create \
--username airflow \
--password airflow \
--firstname Airflow \
--lastname Admin \
--role Admin \
--email admin@airflow.local
```

3.7 Lancer Airflow

Lancer Airflow en arrière-plan :

```
docker compose up -d
```

Vérifier que les conteneurs sont lancés :

```
docker ps
```

3.8 Accéder à l'interface Web

Ouvrir le navigateur à l'adresse suivante :

<http://localhost:8080>

Identifiants :

Utilisateur	Mot de passe
airflow	airflow

4 Premier DAG : comprendre la logique d’Airflow avec PythonOperator

4.1 Objectif

Dans cette partie, vous allez créer un premier DAG très simple.

Ce DAG contient trois tâches :

1. début du pipeline ;
2. traitement ;
3. fin du pipeline.

L’objectif n’est pas encore de traiter des données, mais de comprendre comment Airflow organise les tâches avec PythonOperator.

4.2 Créer le fichier du DAG

Dans le dossier `dags`, créer le fichier suivant :

`mon_premier_dag.py`

Ajouter le code suivant :

```
1 import pendulum
2
3 from airflow import DAG
4 from airflow.operators.python import PythonOperator
5
6
7 def debut_pipeline():
8     print("Debut du pipeline Big Data")
9
10
11 def traitement_pipeline():
12     print("Traitement en cours")
13     print("Simulation d'une etape de traitement de donnees")
14
15
16 def fin_pipeline():
17     print("Fin du pipeline Big Data")
```

```
18
19
20 with DAG(
21     dag_id="mon_premier_dag",
22     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
23     schedule=None,
24     catchup=False,
25     tags=["initiation", "python-operator"],
26 ) as dag:
27
28     debut = PythonOperator(
29         task_id="debut",
30         python_callable=debut_pipeline,
31     )
32
33     traitement = PythonOperator(
34         task_id="traitement",
35         python_callable=traitement_pipeline,
36     )
37
38     fin = PythonOperator(
39         task_id="fin",
40         python_callable=fin_pipeline,
41     )
42
43     debut >> traitement >> fin
```

4.3 Explication

Élément	Signification
dag_id	Nom du DAG affiché dans Airflow.
start_date	Date à partir de laquelle le DAG peut être exécuté.
schedule=None	Le DAG est déclenché manuellement.
catchup=False	Airflow ne rattrape pas les anciennes exécutions.
PythonOperator	Permet d'exécuter une fonction Python.
python_callable	Indique la fonction Python à exécuter.
debut » traitement » fin	Définit l'ordre d'exécution des tâches.

Remarque

Le symbole » signifie que la tâche à gauche doit s'exécuter avant la tâche à droite.

5 Exécuter le premier DAG

Travail demandé

Réaliser les étapes suivantes :

1. Ouvrir l'interface Web d'Airflow.
2. Chercher le DAG `mon_premier_dag`.
3. Activer le DAG.
4. Déclencher le DAG manuellement.
5. Ouvrir la vue **Graph**.
6. Observer l'ordre des tâches.
7. Vérifier que les tâches deviennent vertes.
8. Ouvrir les logs de chaque tâche.

5.1 Interprétation

Si les trois tâches sont vertes, cela signifie que le pipeline s'est exécuté correctement.

Si une tâche est rouge, cela signifie qu'elle a échoué. Il faut alors consulter les logs pour comprendre l'erreur.

6 Deuxième DAG : simulation d'un pipeline Big Data

6.1 Objectif

Dans cette partie, vous allez créer un DAG qui simule un pipeline Big Data classique.

Le pipeline contient les étapes suivantes :

1. ingestion des données ;
2. stockage dans une zone brute ;
3. validation des données ;
4. transformation des données ;
5. traitement analytique ;
6. chargement des résultats ;
7. génération d'un rapport final.

6.2 Créer le fichier du DAG

Dans le dossier `dags`, créer le fichier suivant :

pipeline_big_data_python.py

Ajouter le code suivant :

```
1 import csv
2 import json
3 import os
4 import pendulum
5
6 from airflow import DAG
7 from airflow.operators.python import PythonOperator
8
9
10 DATA_DIR = "/opt/airflow/data"
11 RAW_FILE = f"{DATA_DIR}/ventes_raw.csv"
12 CLEAN_FILE = f"{DATA_DIR}/ventes_clean.csv"
13 RESULT_FILE = f"{DATA_DIR}/resultats_ventes.json"
14 REPORT_FILE = f"{DATA_DIR}/rapport_pipeline.txt"
15
16
17 def ingestion_donnees():
18     """
19     Cette tache simule l'ingestion de donnees depuis une source externe.
20     Dans un vrai projet, la source peut etre une API, une base de donnees,
21     Kafka, un fichier CSV ou un systeme transactionnel.
22     """
23
24     os.makedirs(DATA_DIR, exist_ok=True)
25
26     ventes = [
27         ["id_vente", "ville", "produit", "prix", "quantite"],
28         [1, "Casablanca", "PC", 8000, 2],
29         [2, "Rabat", "Clavier", 300, 5],
30         [3, "Marrakech", "Souris", 150, 10],
31         [4, "Casablanca", "Ecran", 2500, 3],
32         [5, "Tanger", "PC", 8500, 1],
33         [6, "Rabat", "Ecran", 2300, 2],
34     ]
35
36     with open(RAW_FILE, mode="w", newline="", encoding="utf-8") as file:
37         writer = csv.writer(file)
38         writer.writerows(ventes)
39
```

```
40     print(f"Ingestion terminee. Fichier cree : {RAW_FILE}")
41
42
43 def stockage_zone_brute():
44     """
45     Cette tache simule le stockage des donnees dans une zone brute.
46     Dans un vrai pipeline Big Data, cette zone peut etre HDFS, MinIO,
47     Amazon S3 ou un Data Lake.
48     """
49
50     if not os.path.exists(RAW_FILE):
51         raise FileNotFoundError("Le fichier brut n'existe pas.")
52
53     taille = os.path.getsize(RAW_FILE)
54
55     print("Stockage dans la zone brute termine.")
56     print(f"Fichier brut : {RAW_FILE}")
57     print(f"Taille du fichier : {taille} octets")
58
59
60 def validation_donnees():
61     """
62     Cette tache verifie que les donnees existent et que les colonnes
63     attendues sont presentes.
64     """
65
66     if not os.path.exists(RAW_FILE):
67         raise FileNotFoundError("Le fichier de donnees est introuvable.")
68
69     with open(RAW_FILE, mode="r", encoding="utf-8") as file:
70         reader = csv.reader(file)
71         header = next(reader)
72
73     colonnes_attendues = ["id_vente", "ville", "produit", "prix", "quantite"]
74
75     if header != colonnes_attendues:
76         raise ValueError("Le schema du fichier est incorrect.")
77
78     print("Validation terminee avec succes.")
79     print(f"Colonnes detectees : {header}")
80
81
```

```
82 def transformation_donnees():
83     """
84     Cette tache simule le nettoyage et la transformation des donnees.
85     Elle calcule une nouvelle colonne : montant = prix * quantite.
86     """
87
88     lignes_nettoyees = []
89
90     with open(RAW_FILE, mode="r", encoding="utf-8") as input_file:
91         reader = csv.DictReader(input_file)
92
93         for row in reader:
94             prix = float(row["prix"])
95             quantite = int(row["quantite"])
96             montant = prix * quantite
97
98             lignes_nettoyees.append({
99                 "id_vente": row["id_vente"],
100                 "ville": row["ville"],
101                 "produit": row["produit"],
102                 "prix": prix,
103                 "quantite": quantite,
104                 "montant": montant
105             })
106
107     with open(CLEAN_FILE, mode="w", newline="", encoding="utf-8") as output_file:
108         fieldnames = ["id_vente", "ville", "produit", "prix", "quantite", "montant"]
109         writer = csv.DictWriter(output_file, fieldnames=fieldnames)
110         writer.writeheader()
111         writer.writerows(lignes_nettoyees)
112
113     print("Transformation terminee.")
114     print(f"Fichier nettoye genere : {CLEAN_FILE}")
115
116
117 def traitement_analytique():
118     """
119     Cette tache simule un traitement analytique Big Data.
120     Elle calcule le chiffre d'affaires total par ville.
121     """
122
```

```
123     ca_par_ville = {}
124
125     with open(CLEAN_FILE, mode="r", encoding="utf-8") as file:
126         reader = csv.DictReader(file)
127
128         for row in reader:
129             ville = row["ville"]
130             montant = float(row["montant"])
131             ca_par_ville[ville] = ca_par_ville.get(ville, 0) + montant
132
133     with open(RESULT_FILE, mode="w", encoding="utf-8") as file:
134         json.dump(ca_par_ville, file, indent=4, ensure_ascii=False)
135
136     print("Traitement analytique termine.")
137     print("Chiffre d'affaires par ville :")
138     print(ca_par_ville)
139
140
141 def chargement_resultats():
142     """
143     Cette tache simule le chargement des resultats dans une base analytique
144     ou un Data Warehouse.
145     """
146
147     if not os.path.exists(RESULT_FILE):
148         raise FileNotFoundError("Le fichier des resultats est introuvable.")
149
150     print("Chargement des resultats termine.")
151     print(f"Resultats disponibles dans : {RESULT_FILE}")
152
153
154 def generation_rapport():
155     """
156     Cette tache genere un petit rapport final du pipeline.
157     """
158
159     with open(RESULT_FILE, mode="r", encoding="utf-8") as file:
160         resultats = json.load(file)
161
162     with open(REPORT_FILE, mode="w", encoding="utf-8") as report:
163         report.write("Rapport du pipeline Big Data\n")
164         report.write("=====\n\n")
```

```
165
166     for ville, ca in resultats.items():
167         report.write(f"{ville} : {ca} DH\n")
168
169     print("Rapport final genere.")
170     print(f"Fichier rapport : {REPORT_FILE}")
171
172
173 with DAG(
174     dag_id="pipeline_big_data_python",
175     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
176     schedule=None,
177     catchup=False,
178     tags=["big-data", "python-operator", "pipeline"],
179 ) as dag:
180
181     ingestion = PythonOperator(
182         task_id="ingestion_donnees",
183         python_callable=ingestion_donnees,
184     )
185
186     stockage = PythonOperator(
187         task_id="stockage_zone_brute",
188         python_callable=stockage_zone_brute,
189     )
190
191     validation = PythonOperator(
192         task_id="validation_donnees",
193         python_callable=validation_donnees,
194     )
195
196     transformation = PythonOperator(
197         task_id="transformation_donnees",
198         python_callable=transformation_donnees,
199     )
200
201     traitement = PythonOperator(
202         task_id="traitement_analytique",
203         python_callable=traitement_analytique,
204     )
205
206     chargement = PythonOperator(
```



```

207     task_id="chargement_resultats",
208     python_callable=chargement_resultats,
209 )
210
211 rapport = PythonOperator(
212     task_id="generation_rapport",
213     python_callable=generation_rapport,
214 )
215
216 ingestion >> stockage >> validation >> transformation >> traitement >>
    chargement >> rapport

```

6.3 Rôle de chaque tâche

Tâche	Rôle dans un pipeline Big Data
ingestion_donnees	Simule la récupération des données depuis une source externe.
stockage_zone_brute	Simule le stockage des données dans une zone brute du Data Lake.
validation_donnees	Vérifie l'existence du fichier et la structure des colonnes.
transformation_donnees	Nettoie les données et ajoute une colonne calculée.
traitement_analytique	Calcule des indicateurs analytiques.
chargement_resultats	Simule le chargement vers une base analytique ou un Data Warehouse.
generation_rapport	Génère un rapport final exploitable.

Travail demandé

Exécuter le DAG `pipeline_big_data_python`, puis répondre aux questions suivantes :

1. Quelle est la première tâche exécutée ?
2. Quelle est la dernière tâche exécutée ?
3. Quelle tâche crée le fichier CSV brut ?
4. Quelle tâche vérifie le schéma des données ?
5. Quelle tâche calcule le chiffre d'affaires par ville ?
6. Où peut-on voir les messages affichés par les fonctions Python ?

7 Comprendre les logs

Les logs sont très importants dans Airflow.

Ils permettent de comprendre ce qui s'est passé pendant l'exécution d'une tâche.

Dans cette version, les messages ne viennent pas de la commande `echo`, mais de la fonction Python `print()`.

7.1 Exemple

Dans le DAG précédent, la tâche `validation_donnees` exécute :

```
1 print("Validation terminee avec succes.")
```

Le message correspondant doit apparaître dans les logs de la tâche.

Travail demandé

Pour chaque tâche du DAG `pipeline_big_data_python`, consulter les logs et vérifier que le message affiché correspond à l'étape attendue.

8 Planification automatique

Jusqu'à maintenant, les DAGs sont déclenchés manuellement avec :

```
1 schedule=None
```

Pour exécuter automatiquement un DAG chaque jour, on peut utiliser :

```
1 schedule="@daily"
```

Pour exécuter un DAG toutes les heures :

```
1 schedule="@hourly"
```

Pour exécuter un DAG tous les jours à 2h du matin :

```
1 schedule="0 2 * * *"
```

Travail demandé

Modifier le DAG `pipeline_big_data_python` pour utiliser une planification quotidienne avec :

```
schedule="@daily"
```

Observer ensuite dans l'interface Airflow le champ indiquant la prochaine exécution.

Remarque

Après modification d'un fichier DAG, Airflow peut prendre quelques secondes avant de détecter les changements.

9 Créer volontairement une erreur

9.1 Objectif

Dans cette partie, vous allez provoquer volontairement une erreur pour comprendre le comportement d'Airflow.

Avec `PythonOperator`, on peut provoquer une erreur avec l'instruction :

```
1 raise Exception("Message d'erreur")
```

Modifier temporairement la fonction `validation_donnees` comme suit :

```
1 def validation_donnees():  
2     raise Exception("Erreur volontaire : les donnees ne sont pas valides.")
```

Travail demandé

Réaliser les étapes suivantes :

1. Modifier temporairement la fonction `validation_donnees`.
2. Sauvegarder le fichier.
3. Déclencher le DAG.
4. Observer la tâche en erreur dans la vue **Graph**.
5. Consulter les logs de la tâche échouée.
6. Corriger la fonction.
7. Relancer le DAG.

9.2 Interprétation

Si une tâche échoue, les tâches suivantes ne s'exécutent pas si elles dépendent de cette tâche.

Exemple :

ingestion → validation → transformation

Si `validation` échoue, alors `transformation` ne s'exécute pas.

10 Retries : relancer automatiquement une tâche

Dans certains cas, une tâche peut échouer temporairement.

Exemples :

- serveur indisponible ;
- fichier pas encore arrivé ;
- problème réseau ;
- base de données momentanément indisponible.

Airflow peut relancer automatiquement une tâche grâce aux `retries`.

Exemple :

```

1 from datetime import timedelta
2
3 with DAG(
4     dag_id="pipeline_big_data_python",
5     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
6     schedule=None,
7     catchup=False,
8     default_args={
9         "retries": 2,
10        "retry_delay": timedelta(minutes=1),
11    },
12    tags=["big-data", "python-operator"],
13 ) as dag:
```

Cela signifie :

- Airflow relance la tâche en cas d'échec ;
- le nombre maximal de nouvelles tentatives est 2 ;
- Airflow attend 1 minute entre deux tentatives.

11 Dépendances parallèles

Un DAG n'est pas toujours linéaire.

Après une même tâche, plusieurs traitements peuvent être exécutés en parallèle.

Exemple :

$$\text{preparation} \rightarrow \text{validation} \rightarrow \begin{cases} \text{traitement_par_ville} \\ \text{traitement_par_produit} \end{cases} \rightarrow \text{rapport_final}$$

11.1 Créer un DAG parallèle

Dans le dossier `dags`, créer le fichier :

pipeline_big_data_parallele.py

Ajouter le code suivant :

```
1 import csv
2 import json
3 import os
4 import pendulum
5
6 from airflow import DAG
7 from airflow.operators.python import PythonOperator
8
9
10 DATA_DIR = "/opt/airflow/data"
11 CLEAN_FILE = f"{DATA_DIR}/ventes_clean.csv"
12 RESULT_VILLE = f"{DATA_DIR}/resultat_par_ville.json"
13 RESULT_PRODUIT = f"{DATA_DIR}/resultat_par_produit.json"
14 REPORT_FILE = f"{DATA_DIR}/rapport_parallel.txt"
15
16
17 def preparation_donnees():
18     os.makedirs(DATA_DIR, exist_ok=True)
19
20     ventes = [
21         ["id_vente", "ville", "produit", "prix", "quantite", "montant"],
22         [1, "Casablanca", "PC", 8000, 2, 16000],
23         [2, "Rabat", "Clavier", 300, 5, 1500],
24         [3, "Marrakech", "Souris", 150, 10, 1500],
25         [4, "Casablanca", "Ecran", 2500, 3, 7500],
26         [5, "Tanger", "PC", 8500, 1, 8500],
27         [6, "Rabat", "Ecran", 2300, 2, 4600],
28     ]
29
30     with open(CLEAN_FILE, mode="w", newline="", encoding="utf-8") as file:
31         writer = csv.writer(file)
32         writer.writerows(ventes)
33
34     print("Donnees preparees avec succes.")
35
36
37 def validation_donnees():
38     if not os.path.exists(CLEAN_FILE):
39         raise FileNotFoundError("Le fichier nettoye n'existe pas.")
```

```
40
41     print("Validation reussie.")
42
43
44 def traitement_par_ville():
45     resultats = {}
46
47     with open(CLEAN_FILE, mode="r", encoding="utf-8") as file:
48         reader = csv.DictReader(file)
49
50         for row in reader:
51             ville = row["ville"]
52             montant = float(row["montant"])
53             resultats[ville] = resultats.get(ville, 0) + montant
54
55     with open(RESULT_VILLE, mode="w", encoding="utf-8") as file:
56         json.dump(resultats, file, indent=4, ensure_ascii=False)
57
58     print("Traitement par ville termine.")
59     print(resultats)
60
61
62 def traitement_par_produit():
63     resultats = {}
64
65     with open(CLEAN_FILE, mode="r", encoding="utf-8") as file:
66         reader = csv.DictReader(file)
67
68         for row in reader:
69             produit = row["produit"]
70             montant = float(row["montant"])
71             resultats[produit] = resultats.get(produit, 0) + montant
72
73     with open(RESULT_PRODUIT, mode="w", encoding="utf-8") as file:
74         json.dump(resultats, file, indent=4, ensure_ascii=False)
75
76     print("Traitement par produit termine.")
77     print(resultats)
78
79
80 def generation_rapport_final():
81     with open(RESULT_VILLE, mode="r", encoding="utf-8") as file:
```

```

82     par_ville = json.load(file)
83
84     with open(RESULT_PRODUIT, mode="r", encoding="utf-8") as file:
85         par_produit = json.load(file)
86
87     with open(REPORT_FILE, mode="w", encoding="utf-8") as report:
88         report.write("Rapport final du pipeline parallele\n")
89         report.write("=====\n\n")
90
91         report.write("Chiffre d'affaires par ville\n")
92         for ville, ca in par_ville.items():
93             report.write(f"{ville} : {ca} DH\n")
94
95         report.write("\nChiffre d'affaires par produit\n")
96         for produit, ca in par_produit.items():
97             report.write(f"{produit} : {ca} DH\n")
98
99     print("Rapport final genere avec succes.")
100
101
102 with DAG(
103     dag_id="pipeline_big_data_parallele",
104     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
105     schedule=None,
106     catchup=False,
107     tags=["big-data", "parallelisme", "python-operator"],
108 ) as dag:
109
110     preparation = PythonOperator(
111         task_id="preparation_donnees",
112         python_callable=preparation_donnees,
113     )
114
115     validation = PythonOperator(
116         task_id="validation_donnees",
117         python_callable=validation_donnees,
118     )
119
120     analyse_ville = PythonOperator(
121         task_id="traitement_par_ville",
122         python_callable=traitement_par_ville,
123     )

```

```
124
125 analyse_produit = PythonOperator(
126     task_id="traitement_par_produit",
127     python_callable=traitement_par_produit,
128 )
129
130 rapport = PythonOperator(
131     task_id="generation_rapport_final",
132     python_callable=generation_rapport_final,
133 )
134
135 preparation >> validation
136 validation >> [analyse_ville, analyse_produit]
137 [analyse_ville, analyse_produit] >> rapport
```

Travail demandé

Exécuter le DAG `pipeline_big_data_parallele`, puis répondre aux questions suivantes :

1. Quelles tâches sont exécutées avant le parallélisme ?
2. Quelles tâches peuvent s'exécuter en parallèle ?
3. Quelle tâche attend la fin des deux traitements ?
4. Comment le parallélisme est-il représenté dans la vue **Graph** ?

12 Travail à faire

12.1 Contexte

Une administration souhaite automatiser un petit pipeline Big Data pour traiter les inscriptions des étudiants.

Le pipeline doit simuler les étapes suivantes :

1. réception du fichier des étudiants ;
2. stockage du fichier dans une zone brute ;
3. validation du fichier ;
4. nettoyage des données ;
5. affectation des étudiants aux groupes ;
6. génération de statistiques ;
7. génération du rapport final.

12.2 Travail demandé

Travail demandé

Créer un DAG nommé :

`pipeline_inscription_etudiants`

Le DAG doit utiliser uniquement `PythonOperator`.

Il doit contenir au minimum les tâches suivantes :

1. `reception_fichier`
2. `stockage_zone_brute`
3. `validation_fichier`
4. `nettoyage_donnees`
5. `affectation_groupes`
6. `generation_statistiques`
7. `rapport_final`

12.3 Ordre attendu

Le DAG doit respecter l'ordre suivant :

`reception_fichier` → `stockage_zone_brute` → `validation_fichier` →
`nettoyage_donnees`

Après la tâche `nettoyage_donnees`, deux tâches doivent s'exécuter en parallèle :

- `affectation_groupes`
- `generation_statistiques`

La tâche `rapport_final` doit s'exécuter après la fin des deux tâches parallèles.

$$\text{nettoyage_donnees} \rightarrow \begin{cases} \text{affectation_groupes} \\ \text{generation_statistiques} \end{cases} \rightarrow \text{rapport_final}$$

12.4 Consignes

- Chaque tâche doit être une fonction Python.
- Chaque fonction doit afficher un message avec `print()`.
- Le DAG doit être visible dans l'interface Airflow.
- Le DAG doit s'exécuter sans erreur.
- Les logs de chaque tâche doivent être consultables.

12.5 Aide : structure minimale

```
1 import pendulum
2
3 from airflow import DAG
4 from airflow.operators.python import PythonOperator
5
6
7 def reception_fichier():
8     print("Reception du fichier des etudiants")
9
10
11 def stockage_zone_brute():
12     print("Stockage du fichier dans la zone brute")
13
14
15 def validation_fichier():
16     print("Validation du fichier des etudiants")
17
18
19 def nettoyage_donnees():
20     print("Nettoyage des donnees")
21
22
23 def affectation_groupes():
24     print("Affectation des etudiants aux groupes")
25
26
27 def generation_statistiques():
28     print("Generation des statistiques")
29
30
31 def rapport_final():
32     print("Generation du rapport final")
33
34
35 with DAG(
36     dag_id="pipeline_inscription_etudiants",
37     start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
38     schedule=None,
39     catchup=False,
40     tags=["mini-projet", "python-operator"],
```

```
41 ) as dag:
42
43     reception = PythonOperator(
44         task_id="reception_fichier",
45         python_callable=reception_fichier,
46     )
47
48     stockage = PythonOperator(
49         task_id="stockage_zone_brute",
50         python_callable=stockage_zone_brute,
51     )
52
53     validation = PythonOperator(
54         task_id="validation_fichier",
55         python_callable=validation_fichier,
56     )
57
58     nettoyage = PythonOperator(
59         task_id="nettoyage_donnees",
60         python_callable=nettoyage_donnees,
61     )
62
63     affectation = PythonOperator(
64         task_id="affectation_groupes",
65         python_callable=affectation_groupes,
66     )
67
68     statistiques = PythonOperator(
69         task_id="generation_statistiques",
70         python_callable=generation_statistiques,
71     )
72
73     rapport = PythonOperator(
74         task_id="rapport_final",
75         python_callable=rapport_final,
76     )
77
78     reception >> stockage >> validation >> nettoyage
79     nettoyage >> [affectation, statistiques]
80     [affectation, statistiques] >> rapport
```

13 Livrables

Travail demandé

À la fin de l'atelier, rendre les éléments suivants :

1. le fichier `mon_premier_dag.py` ;
2. le fichier `pipeline_big_data_python.py` ;
3. le fichier `pipeline_big_data_parallelele.py` ;
4. le fichier du mini-projet `pipeline_inscription_etudiants.py` ;
5. une capture de la liste des DAGs dans l'interface Airflow ;
6. une capture de la vue **Graph** d'un DAG réussi ;
7. une capture des logs d'une tâche `PythonOperator` ;
8. une courte réponse aux questions de compréhension.

14 Commandes utiles

Commande	Rôle
<code>docker compose up -d</code>	Lancer Airflow en arrière-plan.
<code>docker compose down</code>	Arrêter Airflow.
<code>docker ps</code>	Voir les conteneurs actifs.
<code>docker compose logs airflow-scheduler</code>	Voir les logs du scheduler.
<code>docker compose logs airflow-webserver</code>	Voir les logs du webserver.
<code>docker compose run airflow-webserver airflow dags list</code>	Lister les DAGs.
<code>docker compose run airflow-webserver airflow tasks list mon_premier_dag</code>	Lister les tâches d'un DAG.
<code>docker compose down -volumes -remove-orphans</code>	Supprimer l'environnement Airflow et les volumes associés.